🚀 RTAB-Map Vision and LiDAR Fusion Localization Autonomous Navigation (RTAB-Map Navigation)

1. Course Content

Based on the pre-built `rtabmap.db` 3D map database, enable RTAB-Map's pure localization mode (`localization: true`, `Mem/IncrementalMemory: false`), replacing traditional AMCL single LiDAR Monte Carlo particle filtering, combined with the Nav2 navigation stack to achieve high-precision autonomous obstacle avoidance and path planning.

> **Important**
>
> - This course is only applicable to users with the **Angstrong Nuwa-HP60C Depth Camera** version;
> - Before running this course, please ensure you have successfully saved the map database to `~/map/rtabmap.db` by following the previous lecture "8. RTAB-Map Algorithm SLAM".

---

2. Pure Localization and Navigation Principle

Traditional 2D AMCL relies only on LiDAR single-line cross-section geometric matching, which is prone to longitudinal drift and symmetric position mismatch in long corridors, glass walls, or large open spaces.

RTAB-Map visual relocalization mechanism:

1. **Visual global feature relocalization (Global Relocalization)**: Extract visual features from the current camera frame and perform loop closure comparison with keyframe Bag-of-Words (BoW) in the offline database `rtabmap.db`, recalling the robot's real spatial pose in the map at second level;
2. **Multi-sensor tightly-coupled localization**: Fuse LiDAR scan edge matching and visual photometric geometric constraints, broadcast the `map -> odom` static/dynamic coordinate transform (TF) at high frequency;
3. **Seamless bridge to Nav2 planner**: Shield traditional AMCL node, let RTAB-Map continuously provide global localization source, and Nav2 controller and global planner execute dynamic obstacle avoidance based on visual and LiDAR costmaps.

---

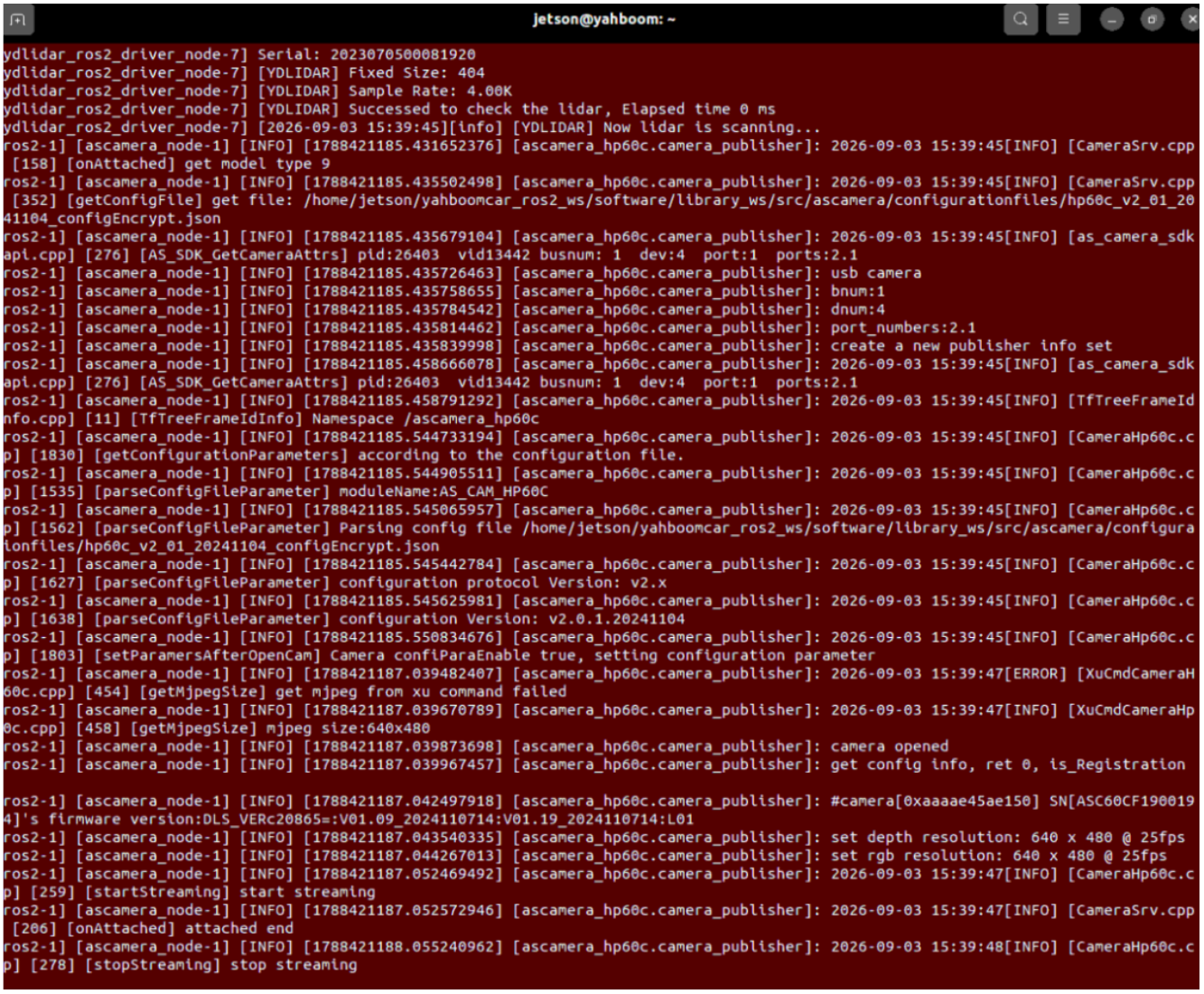
3. Preparation and Running

This case requires opening 3 terminals to start the system in sequence:

3.1 Start Depth Camera Low-level Driver

Run in **Terminal 1** (start Nuwa depth camera, chassis driver, EKF filtering and LiDAR):

```
ros2 launch dofbot_pro_bringup car_base.launch.py start_apriltag_relocalization:=false
```

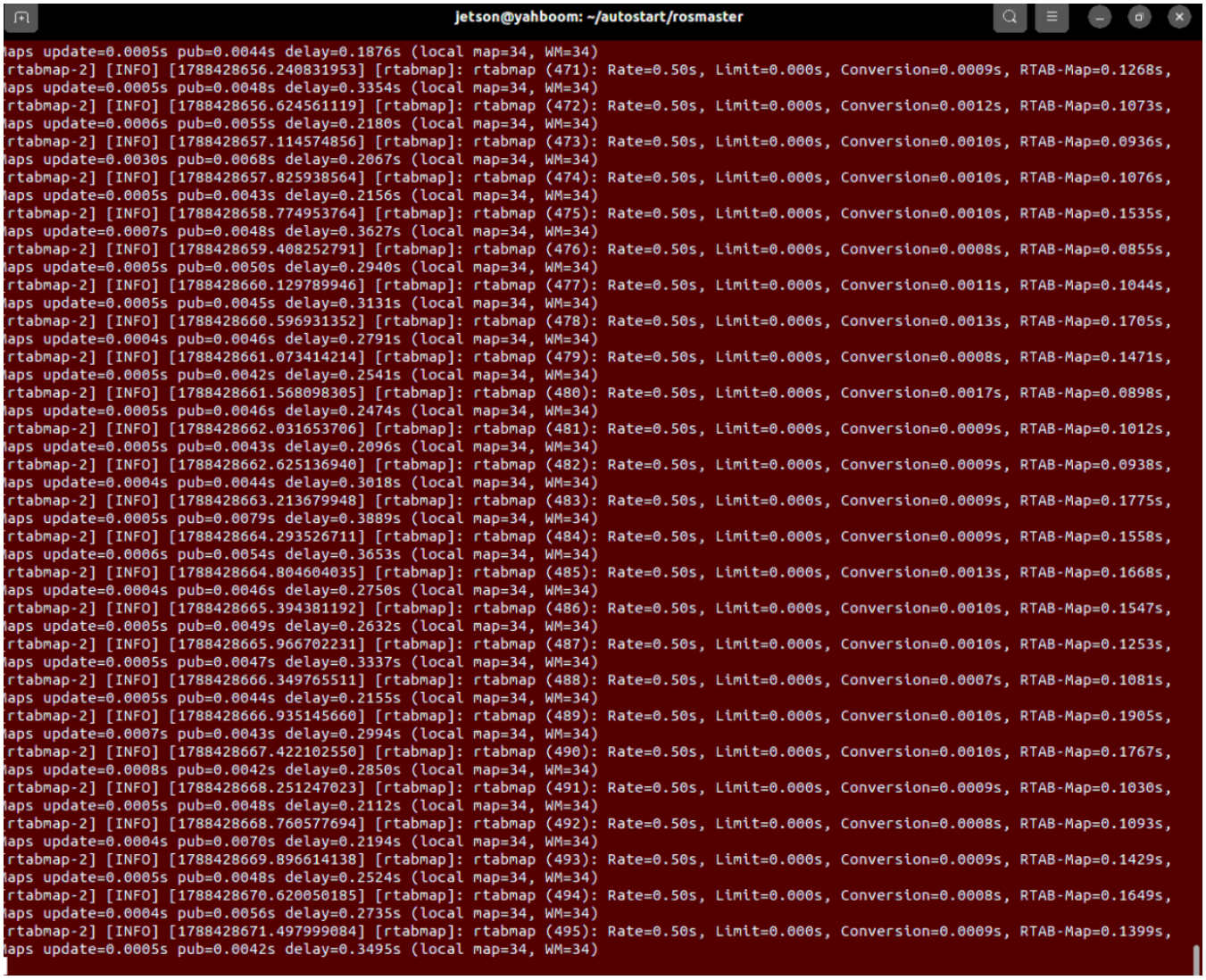



```
# Control robotic arm to look forward
ros2 topic pub /arm6_joints dofbot_pro_interface/msg/ArmJoints "{joint1: 90, joint2: 180,
joint3: 0, joint4: 0, joint5: 90, joint6: 0, time: 1000}" --once
```

3.2 Start RTAB-Map Localization Navigation Core

Run in **Terminal 2**: (It is recommended to start in virtual machine)

```
# Positioning and Navigation Master Control
ros2 launch yahboomcar_nav navigation_rtabmap_launch.py
```

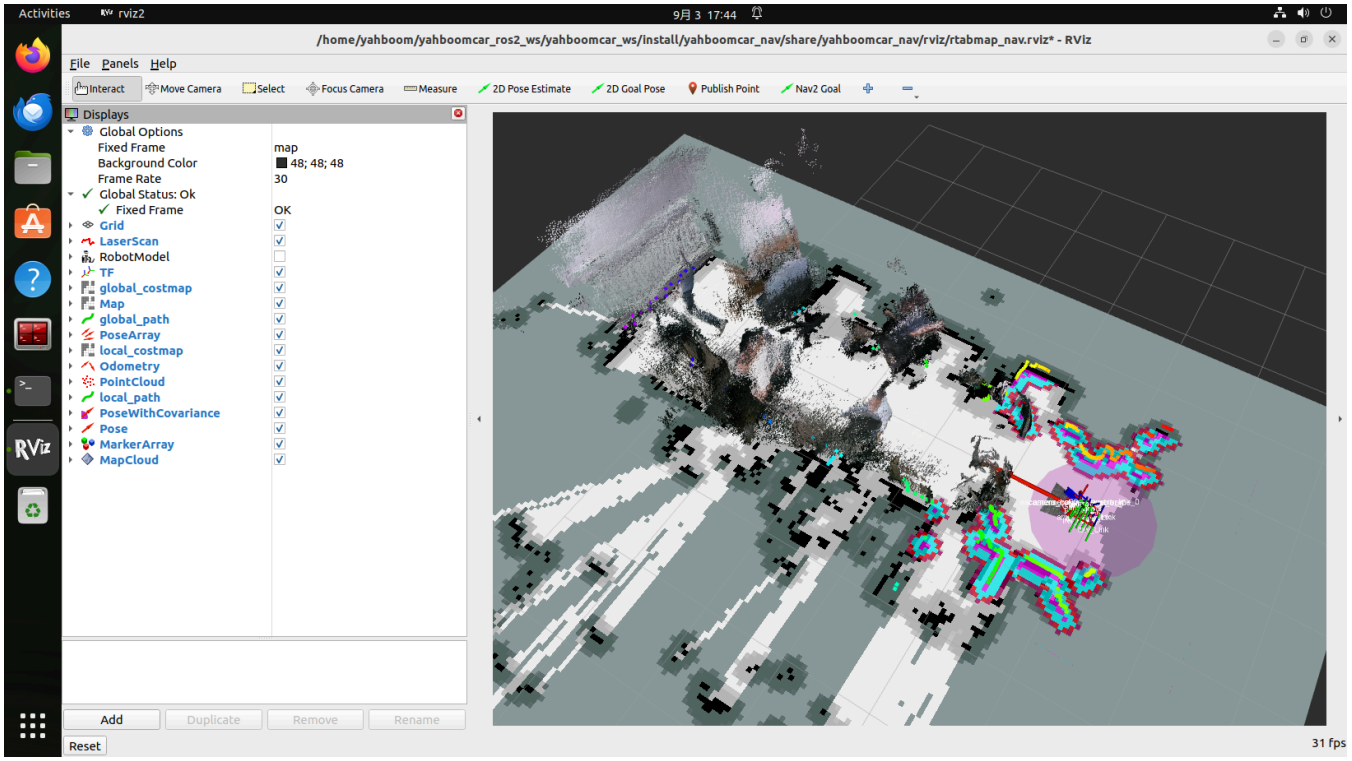


3.3 Start RViz Navigation Visualization Interface

Run in **Terminal 3**:

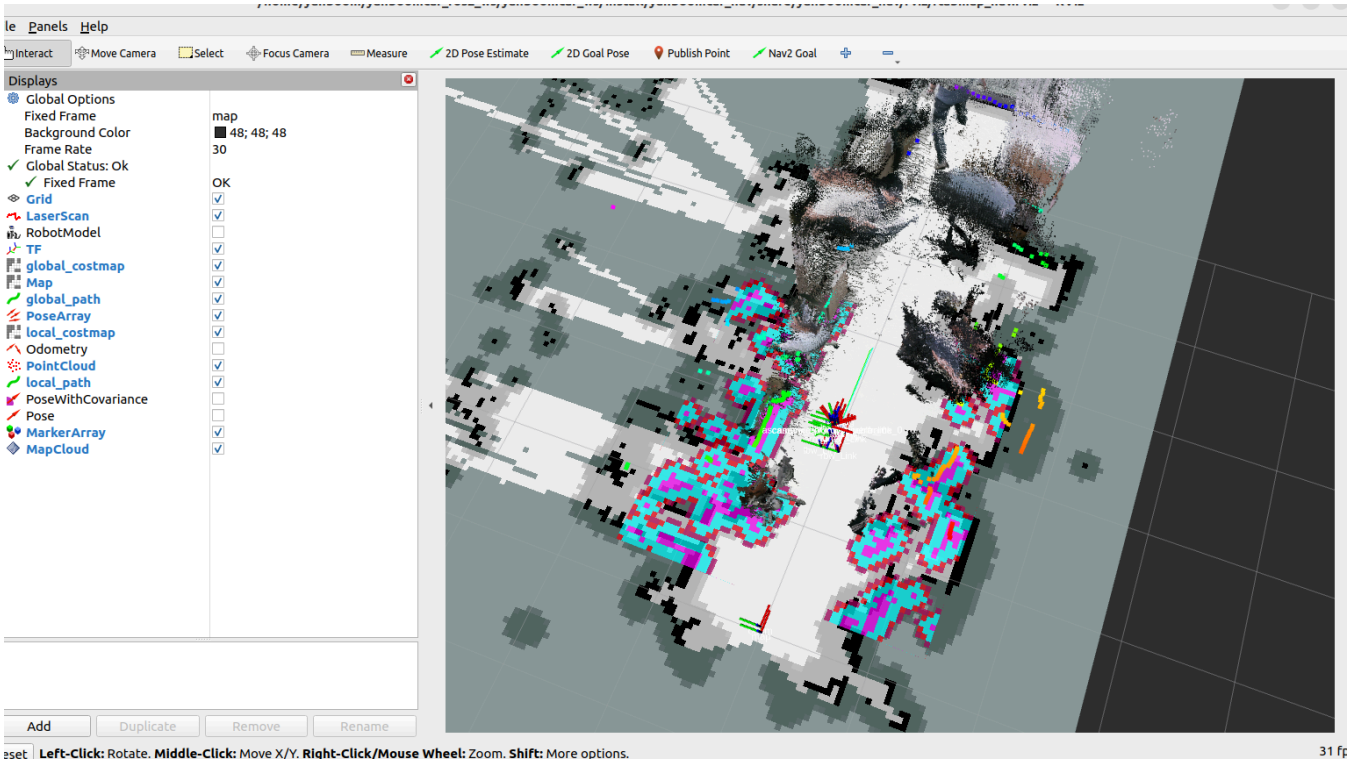
```
ros2 launch yahboomcar_nav display_rtabmap_nav_launch.py
```

The visualization interface will automatically show the loaded global 3D point cloud map and 2D grid costmap:



4. Hands-on Demo and Autonomous Navigation

1. Observe in the RViz interface that the 3D offline point cloud map and 2D costmap are successfully loaded;
2. **Relocalization initialization**: If the robot's initial pose is not fully aligned with the map, you can use the `2D Pose Estimate` tool in the RViz top toolbar, drag a green arrow along the actual physical position and heading direction of the robot to help the algorithm quickly lock the position;
3. **Send navigation goal point**: Click the `2D Goal Pose` (or Nav2 Goal) tool in the RViz top toolbar, specify any target point on the map and pull out the desired heading;
4. The robot will autonomously plan the globally optimal path, combined with Nav2 local obstacle avoidance to dynamically bypass obstacles and smoothly travel to the destination:



5. Core Architecture and Source Code Implementation Analysis

Source Code Reference:

- `yahboomcar_ws/src/yahboomcar_nav/launch/rtab/navigation_rtabmap_launch.py`
- `yahboomcar_ws/src/yahboomcar_nav/launch/rtab/rtabmap_localization_launch.py`
- `yahboomcar_ws/src/yahboomcar_nav/launch/rtab/rtabmap_nav_launch.py`

The core logic of the entire localization navigation system is the **"RTAB-Map is responsible for high-precision global localization (map -> odom) + Nav2 is responsible for path planning and motion control (cmd_vel)"** dual-engine collaborative working mode:

5.1 Main Control Scheduling Node (`navigation_rtabmap_launch.py`)

The system top-level simultaneously pulls up the localization layer and navigation layer through modular composition:

```
# navigation_rtabmap_launch.py
def generate_launch_description():
    rtabmap_launch_dir = os.path.join(
        get_package_share_directory('yahboomcar_nav'), 'launch', 'rtab')

    # 1. Start RTAB-Map pure localization node
    rtabmap_localization_launch = IncludeLaunchDescription(
        PythonLaunchDescriptionSource(
            os.path.join(rtabmap_launch_dir, 'rtabmap_localization_launch.py'))
    )
```

```
    # 2. Start Nav2 navigation planner
    nav2_launch = IncludeLaunchDescription(
        PythonLaunchDescriptionSource(
            os.path.join(rtabmap_launch_dir, 'rtabmap_nav_launch.py'))
    )

    return LaunchDescription([
        rtabmap_localization_launch,
        nav2_launch,
    ])
```

5.2 Frozen Mapping Pure Localization Mode Configuration

In `rtabmap_localization_launch.py`, the key parameter is to turn off memory incremental growth, lock the database, and only perform matching relocalization while continuously broadcasting `map -> odom` TF:

```
# rtabmap_localization_launch.py —— Pure localization core configuration
parameters = {
    'Mem/IncrementalMemory': 'false',  # Key: Completely turn off incremental mapping, freeze node updates
    'Mem/InitWMWithAllNodes': 'true',  # Load all keyframes from database to working memory during initialization
    'RGBD/SavedLocalizationIgnored': 'true',
    'database_path': '~/map/rtabmap.db',
}
```

5.3 Nav2 Navigation Base Integration (`rtabmap_nav_launch.py`)

`rtabmap_nav_launch.py` calls the standard `nav2_bringup`'s `navigation_launch.py`, loading the costmap and controller parameter files `rtabmap_nav_params.yaml` specifically tuned for RTAB-Map:

```
# rtabmap_nav_launch.py
nav2_navigation_launch = os.path.join(
    get_package_share_directory('nav2_bringup'), 'launch', 'navigation_launch.py')

# Pass rtabmap_nav_params.yaml configuration, cooperate with costmap for obstacle detection and trajectory tracking
```